

Directed Acyclic Graphs

Sean

August 27, 2026



Outline

- Why DAGs appear
- Topological sorting
- Kahn's algorithm
- DFS viewpoint
- Dynamic programming on DAGs
- Problem patterns



Why another graph family?

- Some directed graphs describe **dependencies**: one task must finish before another can start



Why another graph family?

- Some directed graphs describe **dependencies**: one task must finish before another can start
- Examples: course prerequisites, build systems, recipe steps, project schedules



Why another graph family?

- Some directed graphs describe **dependencies**: one task must finish before another can start
- Examples: course prerequisites, build systems, recipe steps, project schedules
- In these problems, cycles are not just inconvenient—they make the plan impossible



Why another graph family?

- Some directed graphs describe **dependencies**: one task must finish before another can start
- Examples: course prerequisites, build systems, recipe steps, project schedules
- In these problems, cycles are not just inconvenient—they make the plan impossible
- DAGs are exactly the directed graphs where dependency order makes sense

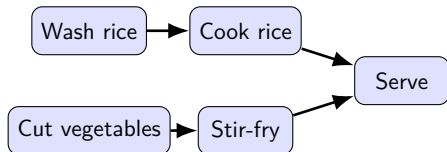


- 1 Why DAGs appear
- 2 Topological sorting
- 3 Kahn's algorithm
- 4 DFS viewpoint
- 5 Dynamic programming on DAGs
- 6 Problem patterns



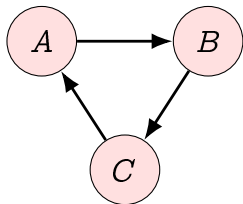
Why DAGs appear

A dependency graph



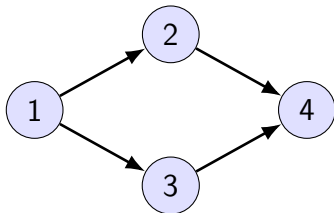
- Vertices are tasks
- Edge $u \rightarrow v$: do u before v
- Several valid schedules may exist

What goes wrong with a cycle?



- *A* waits for *C*
- *C* waits for *B*
- *B* waits for *A*
- No task can be first

DAG = directed + acyclic



- **Directed:** edges have a direction
- **Acyclic:** no directed cycle exists
- Every edge points “forward” in some valid order

- 1 Why DAGs appear
- 2 Topological sorting
- 3 Kahn's algorithm
- 4 DFS viewpoint
- 5 Dynamic programming on DAGs
- 6 Problem patterns



Topological sorting

Topological sorting

Definition

A **topological order** of a directed graph is an ordering of the vertices such that every edge $u \rightarrow v$ places u before v .

Topological sorting

Definition

A **topological order** of a directed graph is an ordering of the vertices such that every edge $u \rightarrow v$ places u before v .

- A **topological sort** is the process of finding one such order



Topological sorting

Definition

A **topological order** of a directed graph is an ordering of the vertices such that every edge $u \rightarrow v$ places u before v .

- A **topological sort** is the process of finding one such order
- It is a valid schedule for all dependencies



Topological sorting

Definition

A **topological order** of a directed graph is an ordering of the vertices such that every edge $u \rightarrow v$ places u before v .

- A **topological sort** is the process of finding one such order
- It is a valid schedule for all dependencies
- A directed graph has a topological order **if and only if** it is acyclic



Topological sorting

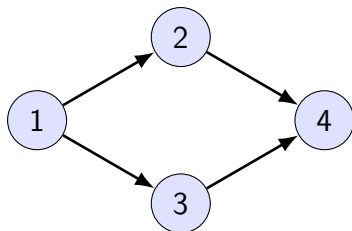
Definition

A **topological order** of a directed graph is an ordering of the vertices such that every edge $u \rightarrow v$ places u before v .

- A **topological sort** is the process of finding one such order
- It is a valid schedule for all dependencies
- A directed graph has a topological order **if and only if** it is acyclic
- The order is often not unique

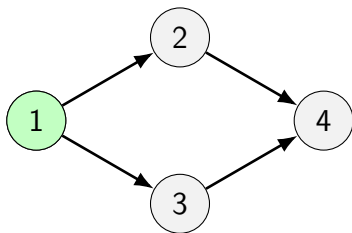


One graph, many valid orders



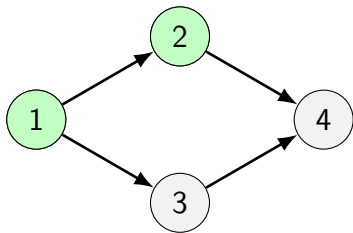
- Valid: 1, 2, 3, 4
- Also valid: 1, 3, 2, 4
- Invalid: 2, 1, 3, 4
because $1 \rightarrow 2$

Reading a topological order



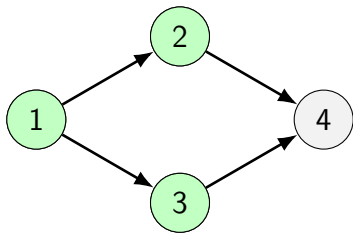
- Start with a vertex that has no unmet prerequisite

Reading a topological order



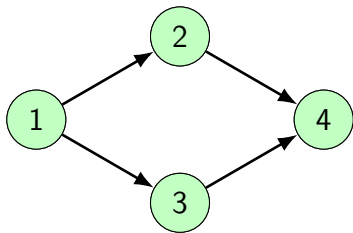
- Start with a vertex that has no unmet prerequisite
- After placing 1, vertex 2 becomes available

Reading a topological order



- Start with a vertex that has no unmet prerequisite
- After placing 1, vertex 2 becomes available
- Vertex 3 is also available

Reading a topological order



- Start with a vertex that has no unmet prerequisite
- After placing 1, vertex 2 becomes available
- Vertex 3 is also available
- Only then can 4 appear

- 1 Why DAGs appear
- 2 Topological sorting
- 3 Kahn's algorithm**
- 4 DFS viewpoint
- 5 Dynamic programming on DAGs
- 6 Problem patterns



Kahn's algorithm

The indegree idea

- The **indegree** of a vertex is how many incoming edges it has

The indegree idea

- The **indegree** of a vertex is how many incoming edges it has
- A vertex with indegree 0 has no unmet prerequisite



The indegree idea

- The **indegree** of a vertex is how many incoming edges it has
- A vertex with indegree 0 has no unmet prerequisite
- So it can safely be placed next in a topological order



The indegree idea

- The **indegree** of a vertex is how many incoming edges it has
- A vertex with indegree 0 has no unmet prerequisite
- So it can safely be placed next in a topological order
- After removing it, some neighbors may become indegree 0



Kahn's algorithm

- Compute every indegree

Kahn's algorithm

- Compute every indegree
- Put all indegree-0 vertices into a queue



Kahn's algorithm

- Compute every indegree
- Put all indegree-0 vertices into a queue
- Repeatedly pop one vertex, append it to the order, and remove its outgoing edges



Kahn's algorithm

- Compute every indegree
- Put all indegree-0 vertices into a queue
- Repeatedly pop one vertex, append it to the order, and remove its outgoing edges
- Whenever a neighbor's indegree becomes 0, push it into the queue



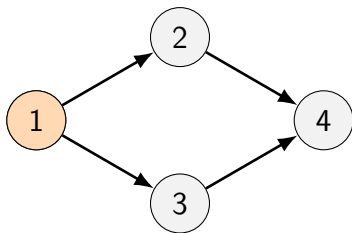
Kahn's algorithm

- Compute every indegree
- Put all indegree-0 vertices into a queue
- Repeatedly pop one vertex, append it to the order, and remove its outgoing edges
- Whenever a neighbor's indegree becomes 0, push it into the queue
- If fewer than n vertices are processed, a cycle exists

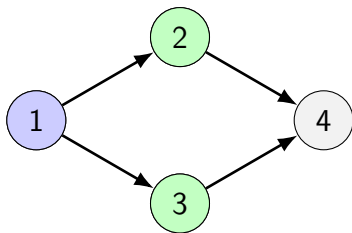


Kahn trace

- Start: queue [1]

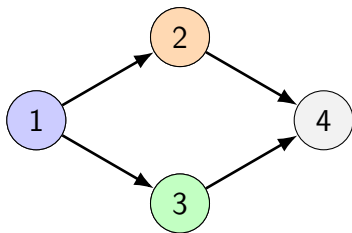


Kahn trace



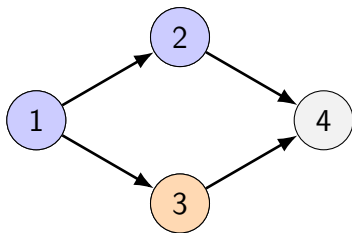
- Start: queue $[1]$
- Remove 1: queue $[2, 3]$

Kahn trace



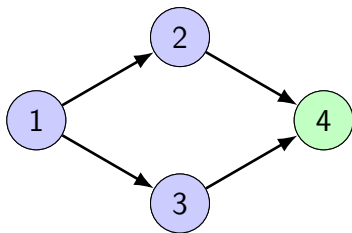
- Start: queue [1]
- Remove 1: queue [2, 3]
- Remove 2: queue [3]

Kahn trace



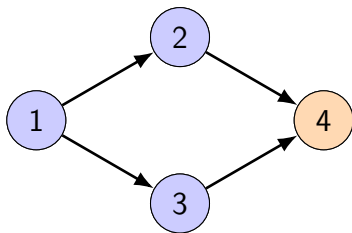
- Start: queue [1]
- Remove 1: queue [2, 3]
- Remove 2: queue [3]
- Remove 3: queue [4]

Kahn trace



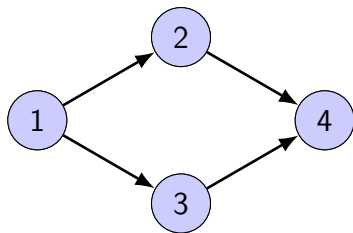
- Start: queue [1]
- Remove 1: queue [2, 3]
- Remove 2: queue [3]
- Remove 3: queue [4]
- Vertex 4 is now ready

Kahn trace



- Start: queue [1]
- Remove 1: queue [2, 3]
- Remove 2: queue [3]
- Remove 3: queue [4]
- Vertex 4 is now ready
- Remove 4

Kahn trace



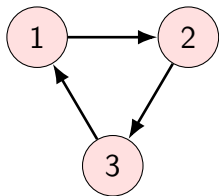
- Start: queue [1]
- Remove 1: queue [2, 3]
- Remove 2: queue [3]
- Remove 3: queue [4]
- Vertex 4 is now ready
- Remove 4
- Order: 1, 2, 3, 4

Kahn code

```
1  vector<int> indeg(n + 1, 0);
2  for (int u = 1; u <= n; u++) {
3      for (int v : adj[u]) indeg[v]++;
4  }
5
6  queue<int> q;
7  for (int u = 1; u <= n; u++) {
8      if (indeg[u] == 0) q.push(u);
9  }
10 vector<int> order;
11 while (!q.empty()) {
12     int u = q.front();
13     q.pop();
14     order.push_back(u);
15
16     for (int v : adj[u]) {
17         if (--indeg[v] == 0) q.push(v);
18     }
19 }
20 if ((int)order.size() != n) {
21     // A cycle exists: no topological order was found.
22 }
```



Detecting a cycle



- Every vertex has indegree 1
- Initial queue is empty
- Processed count is $0 < n$
- Therefore a directed cycle exists

Example: Course Schedule

CSES: Course Schedule

Given course prerequisites, output one valid order or report that it is impossible.

Example: Course Schedule

CSES: Course Schedule

Given course prerequisites, output one valid order or report that it is impossible.

- Vertices are courses



Example: Course Schedule

CSES: Course Schedule

Given course prerequisites, output one valid order or report that it is impossible.

- Vertices are courses
- Edge $a \rightarrow b$: take a before b



Example: Course Schedule

CSES: Course Schedule

Given course prerequisites, output one valid order or report that it is impossible.

- Vertices are courses
- Edge $a \rightarrow b$: take a before b
- Kahn's algorithm builds a schedule



Example: Course Schedule

CSES: Course Schedule

Given course prerequisites, output one valid order or report that it is impossible.

- Vertices are courses
- Edge $a \rightarrow b$: take a before b
- Kahn's algorithm builds a schedule
- If the order length is smaller than n , output IMPOSSIBLE



- 1 Why DAGs appear
- 2 Topological sorting
- 3 Kahn's algorithm
- 4 DFS viewpoint**
- 5 Dynamic programming on DAGs
- 6 Problem patterns



DFS viewpoint

DFS gives another order

- In DFS, a vertex finishes only after all reachable descendants are finished

DFS gives another order

- In DFS, a vertex finishes only after all reachable descendants are finished
- Therefore, if we record vertices on exit, descendants appear before ancestors

DFS gives another order

- In DFS, a vertex finishes only after all reachable descendants are finished
- Therefore, if we record vertices on exit, descendants appear before ancestors
- Reversing that finish order gives a topological order

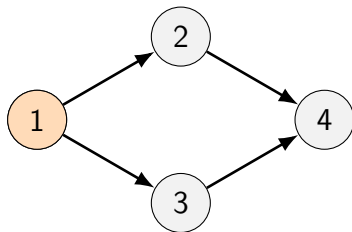


DFS gives another order

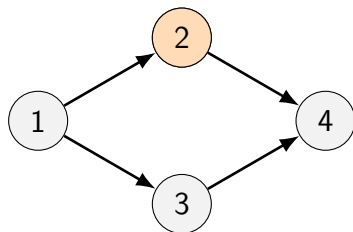
- In DFS, a vertex finishes only after all reachable descendants are finished
- Therefore, if we record vertices on exit, descendants appear before ancestors
- Reversing that finish order gives a topological order
- A back edge to an active vertex reveals a directed cycle

DFS finish-order trace

- Enter 1

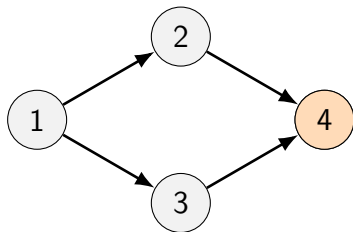


DFS finish-order trace



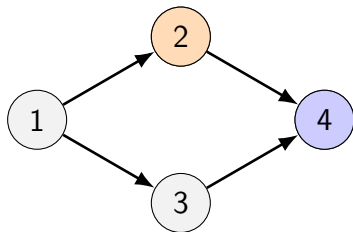
- Enter 1
- Explore 2

DFS finish-order trace



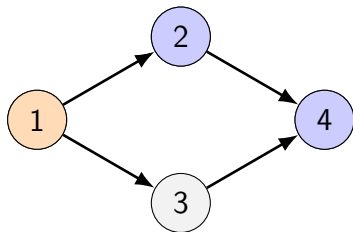
- Enter 1
- Explore 2
- Explore 4

DFS finish-order trace



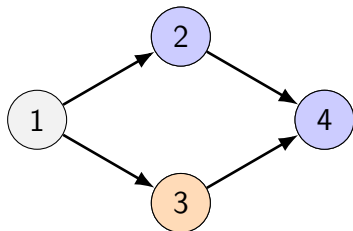
- Enter 1
- Explore 2
- Explore 4
- Finish 4

DFS finish-order trace



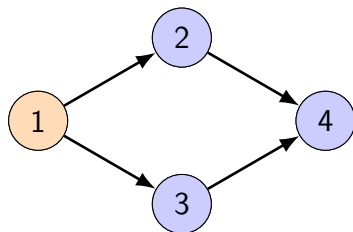
- Enter 1
- Explore 2
- Explore 4
- Finish 4
- Finish 2

DFS finish-order trace



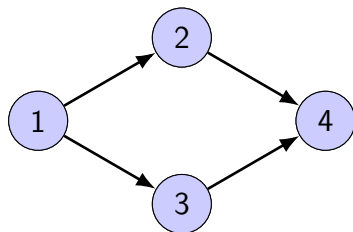
- Enter 1
- Explore 2
- Explore 4
- Finish 4
- Finish 2
- Explore 3

DFS finish-order trace



- Enter 1
- Explore 2
- Explore 4
- Finish 4
- Finish 2
- Explore 3
- Finish 3

DFS finish-order trace



- Enter 1
- Explore 2
- Explore 4
- Finish 4
- Finish 2
- Explore 3
- Finish 3
- Finish 1

- 1 Why DAGs appear
- 2 Topological sorting
- 3 Kahn's algorithm
- 4 DFS viewpoint
- 5 Dynamic programming on DAGs
- 6 Problem patterns



Dynamic programming on DAGs

Why DAGs unlock DP

- DP needs subproblems to be solved before larger problems use them

Why DAGs unlock DP

- DP needs subproblems to be solved before larger problems use them
- A topological order gives exactly that dependency order



Why DAGs unlock DP

- DP needs subproblems to be solved before larger problems use them
- A topological order gives exactly that dependency order
- Process vertices from left to right in topological order



Why DAGs unlock DP

- DP needs subproblems to be solved before larger problems use them
- A topological order gives exactly that dependency order
- Process vertices from left to right in topological order
- Each edge relaxes information forward once its source is already known



Longest path in a DAG

- In general graphs, longest simple path is difficult because avoiding cycles couples choices

Longest path in a DAG

- In general graphs, longest simple path is difficult because avoiding cycles couples choices
- In a DAG, cycles do not exist

Longest path in a DAG

- In general graphs, longest simple path is difficult because avoiding cycles couples choices
- In a DAG, cycles do not exist
- So we can compute longest paths with dynamic programming

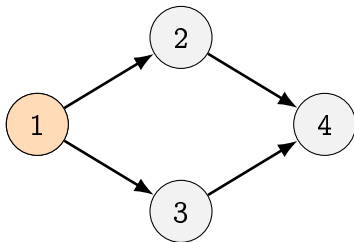


Longest path in a DAG

- In general graphs, longest simple path is difficult because avoiding cycles couples choices
- In a DAG, cycles do not exist
- So we can compute longest paths with dynamic programming
- If $u \rightarrow v$, then $dp[v] = \max(dp[v], dp[u] + 1)$

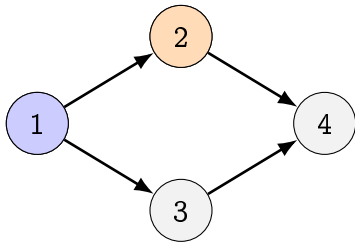


DAG DP trace



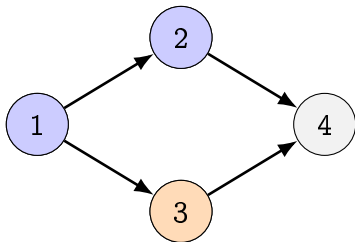
- Start: $dp[1] = 0$

DAG DP trace



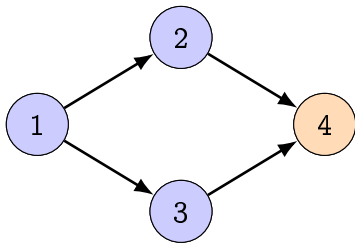
- Start: $dp[1] = 0$
- From 1: $dp[2] = 1$, $dp[3] = 1$

DAG DP trace



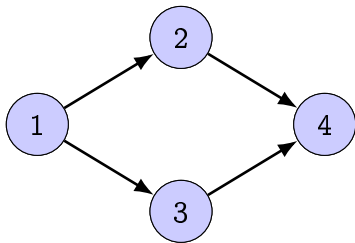
- Start: $dp[1] = 0$
- From 1: $dp[2] = 1$, $dp[3] = 1$
- From 2: $dp[4] = 2$

DAG DP trace



- Start: $dp[1] = 0$
- From 1: $dp[2] = 1$, $dp[3] = 1$
- From 2: $dp[4] = 2$
- From 3: $dp[4]$ stays 2

DAG DP trace



- Start: $dp[1] = 0$
- From 1: $dp[2] = 1$, $dp[3] = 1$
- From 2: $dp[4] = 2$
- From 3: $dp[4]$ stays 2
- Longest path length to 4 is 2

Longest-path DP code

```
1  const int NEG_INF = -1e9;
2  vector<int> dp(n + 1, NEG_INF);
3  dp[start] = 0;
4
5  for (int u : topo) {
6      if (dp[u] == NEG_INF) continue;
7      for (int v : adj[u]) {
8          dp[v] = max(dp[v], dp[u] + 1);
9      }
10 }
```



Example: Longest Flight Route

CSES: Longest Flight Route

Find a route from city 1 to city n with as many cities as possible in a DAG.

Example: Longest Flight Route

CSES: Longest Flight Route

Find a route from city 1 to city n with as many cities as possible in a DAG.

- Use a topological order

Example: Longest Flight Route

CSES: Longest Flight Route

Find a route from city 1 to city n with as many cities as possible in a DAG.

- Use a topological order
- Let $\text{dp}[v]$ be the maximum number of edges on a route to v



Example: Longest Flight Route

CSES: Longest Flight Route

Find a route from city 1 to city n with as many cities as possible in a DAG.

- Use a topological order
- Let $dp[v]$ be the maximum number of edges on a route to v
- Store $parent[v]$ when improving $dp[v]$



Example: Longest Flight Route

CSES: Longest Flight Route

Find a route from city 1 to city n with as many cities as possible in a DAG.

- Use a topological order
- Let $dp[v]$ be the maximum number of edges on a route to v
- Store $parent[v]$ when improving $dp[v]$
- Reconstruct the chosen route at the end



Counting paths in a DAG

- Sometimes we want the **number** of valid routes, not just one best route

Counting paths in a DAG

- Sometimes we want the **number** of valid routes, not just one best route
- Let $ways[v]$ be the number of paths from the start to v



Counting paths in a DAG

- Sometimes we want the **number** of valid routes, not just one best route
- Let $ways[v]$ be the number of paths from the start to v
- Process vertices in topological order



Counting paths in a DAG

- Sometimes we want the **number** of valid routes, not just one best route
- Let $ways[v]$ be the number of paths from the start to v
- Process vertices in topological order
- For every edge $u \rightarrow v$, add $ways[u]$ into $ways[v]$



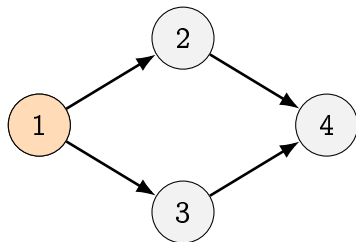
Counting paths in a DAG

- Sometimes we want the **number** of valid routes, not just one best route
- Let $ways[v]$ be the number of paths from the start to v
- Process vertices in topological order
- For every edge $u \rightarrow v$, add $ways[u]$ into $ways[v]$
- In code: $ways[v] += ways[u]$

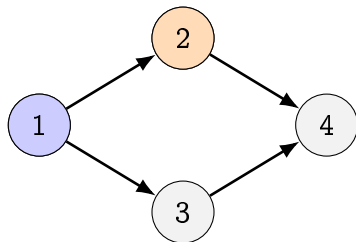


Counting-paths trace

- Start: $ways[1] = 1$

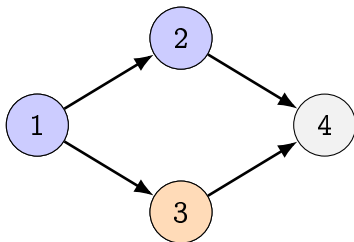


Counting-paths trace



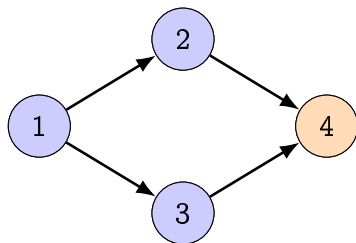
- Start: $ways[1] = 1$
- From 1: $ways[2] = 1$, $ways[3] = 1$

Counting-paths trace



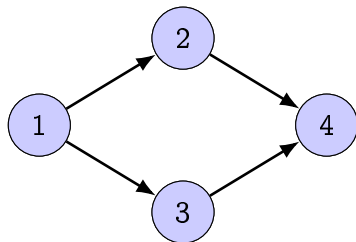
- Start: $ways[1] = 1$
- From 1: $ways[2] = 1$, $ways[3] = 1$
- From 2: $ways[4] = 1$

Counting-paths trace



- Start: $ways[1] = 1$
- From 1: $ways[2] = 1$, $ways[3] = 1$
- From 2: $ways[4] = 1$
- From 3: $ways[4] = 2$

Counting-paths trace



- Start: $ways[1] = 1$
- From 1: $ways[2] = 1$, $ways[3] = 1$
- From 2: $ways[4] = 1$
- From 3: $ways[4] = 2$
- There are 2 paths from 1 to 4

Counting-paths code

```
1  const int MOD = 1e9 + 7;
2  vector<int> ways(n + 1, 0);
3  ways[start] = 1;
4
5  for (int u : topo) {
6      for (int v : adj[u]) {
7          ways[v] = (ways[v] + ways[u]) % MOD;
8      }
9  }
```



Example: Game Routes

CSES: Game Routes

Count how many routes lead from level 1 to level n in a directed acyclic graph.

Example: Game Routes

CSES: Game Routes

Count how many routes lead from level 1 to level n in a directed acyclic graph.

- Let $\text{ways}[v]$ be the number of routes reaching v



Example: Game Routes

CSES: Game Routes

Count how many routes lead from level 1 to level n in a directed acyclic graph.

- Let $\text{ways}[v]$ be the number of routes reaching v
- Start with $\text{ways}[1] = 1$



Example: Game Routes

CSES: Game Routes

Count how many routes lead from level 1 to level n in a directed acyclic graph.

- Let $\text{ways}[v]$ be the number of routes reaching v
- Start with $\text{ways}[1] = 1$
- Push counts forward in topological order



Example: Game Routes

CSES: Game Routes

Count how many routes lead from level 1 to level n in a directed acyclic graph.

- Let $\text{ways}[v]$ be the number of routes reaching v
- Start with $\text{ways}[1] = 1$
- Push counts forward in topological order
- Take answers modulo $10^9 + 7$



- 1 Why DAGs appear
- 2 Topological sorting
- 3 Kahn's algorithm
- 4 DFS viewpoint
- 5 Dynamic programming on DAGs
- 6 Problem patterns**



Problem patterns

Recognizing DAG problems

- The input describes prerequisites, precedence, or one-way progress



Recognizing DAG problems

- The input describes prerequisites, precedence, or one-way progress
- You need a valid ordering, or need to decide whether one exists



Recognizing DAG problems

- The input describes prerequisites, precedence, or one-way progress
- You need a valid ordering, or need to decide whether one exists
- Values should flow from earlier states to later states



Recognizing DAG problems

- The input describes prerequisites, precedence, or one-way progress
- You need a valid ordering, or need to decide whether one exists
- Values should flow from earlier states to later states
- The graph has no reason to revisit a state once progress is made



Which tool should I reach for?

Need	Use	Why
Build any valid order	Topological sort	Respect dependencies
Detect impossible schedule	Kahn or DFS coloring	Cycles block ordering
Longest / count / combine paths	DAG DP	Topological order solves subproblems first
Lexicographically smallest order	Min-heap Kahn	Choose the smallest ready vertex



Common mistakes

- Using BFS order and assuming it is topological order
- Forgetting that many valid topological orders may exist
- Running DAG DP before confirming the graph is acyclic
- Initializing DP incorrectly for unreachable vertices



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

